Anime World Shop - API Documentation

Interactive version: run the backend and open http://localhost:5000/api/docs (Swagger UI).

JSON REST API for the Anime World Shop mobile app and its admin panel.

Every response has the same shape: `{success, message, data}`. Errors have `success: false`, a `message`, and for validation errors an `errors` object saying which field is wrong.

Log in with `POST /api/auth/login`, then press **Authorize** and paste the token. Demo accounts: `demo@shop.com` / `demo123` (customer) and `admin@shop.com` / `demo123` (admin).

Base URL: `http://localhost:5000` (Android emulator: `http://10.0.2.2:5000`). Logged-in routes need the header `Authorization: Bearer <token>`.

Status codes

Code	Meaning
200 / 201	Done (201 = created)
400	Bad request, e.g. an expired code
401	No token, or the token is wrong or expired
403	Not allowed (blocked account, not shop staff, or a staff account on an owner-only route)
404	Not found, or not yours
409	Not possible right now, e.g. sold out or a wrong order step
422	Some fields are wrong; `errors` says which
429	Too many tries; wait and try again
502	The payment provider refused or could not be reached
500	Unexpected server error

Endpoints

Auth

Register, log in, forgot password

Method	Path	What it does	Login
`GET`	`/api/health`	Is the server up?	-
`POST`	`/api/auth/register`	Create a customer account	-
`POST`	`/api/auth/login`	Log in	-
`GET`	`/api/auth/me`	The logged-in user	yes
`POST`	`/api/auth/forgot-password`	Email a 6-digit reset code	-
`POST`	`/api/auth/verify-otp`	Check the code, get a reset token	-
`POST`	`/api/auth/reset-password`	Set a new password with the reset token	-

GET /api/health

Is the server up?.

Response 200: `data` = {status: string}

POST /api/auth/register

Create a customer account.

Also adds a WELCOME10 promo notification. Name 2-100 characters with letters, email up to 150, password 6-72, phone Cambodian. 20 sign-ups an hour per network (429).

Body (application/json): {full_name: string, email: string, phone: string, password: string}

```
{
  "full_name": "Sok Dara",
  "email": "dara@example.com",
  "phone": "012345678",
  "password": "secret123"
}
```

Response 201: `data` = {token: string, user: `User`}

Errors: 422, 429

POST /api/auth/login

Log in.

401 for a wrong email or password, 403 for a blocked account. Tokens last 7 days. 429 after 10 tries a minute from one network, or 10 in 15 minutes for one email.

Body (application/json): {email: string, password: string}

```
{
  "email": "demo@shop.com",
  "password": "demo123"
}
```

Response 200: `data` = {token: string, user: `User`}

Errors: 401, 403, 422, 429

GET /api/auth/me

The logged-in user.

Response 200: `data` = `User`

Errors: 401, 403, 404

POST /api/auth/forgot-password

Email a 6-digit reset code.

Always answers the same, so nobody can find out which emails are registered. 60 s between codes, 5 an hour.

Body (application/json): {email: string}

```
{
  "email": "demo@shop.com"
}
```

Response 200: `data` = {expires_in: integer, resend_after: integer, dev_code: string}

Errors: 422, 429

POST /api/auth/verify-otp

Check the code, get a reset token.

5 wrong tries lock the code.

Body (application/json): {email: string, code: string}

```
{
  "email": "demo@shop.com",
  "code": "123456"
}
```

Response 200: `data` = {reset_token: string}

Errors: 400, 422, 429

POST /api/auth/reset-password

Set a new password with the reset token.

Body (application/json): {reset_token: string, new_password: string}

Response 200: `data` = any or null

Errors: 400, 422

Catalog

Categories, series, brands, products and search (no login needed)

Method	Path	What it does	Login
GET	/api/categories	Categories with product counts and a cover photo	-
GET	/api/series	Anime series that have products, most first	-
GET	/api/brands	Brand names, A to Z	-
GET	/api/search/suggestions	Suggestions while typing	-
GET	/api/products	Search, filter, sort and page products	-
GET	/api/products/{product_id}	Product details with reviews and related products	-

GET /api/categories

Categories with product counts and a cover photo.

Response 200: `data` = list of `Category`

GET /api/series

Anime series that have products, most first.

Response 200: `data` = list of `Series`

GET /api/brands

Brand names, A to Z.

Response 200: `data` = list of string

GET /api/search/suggestions

Suggestions while typing.

Up to 5 products, matching search phrases and the number of matches.

Parameters: `q` (query, required) - What the customer typed.

Response 200: `data` = {products: list of `Product`, keywords: list of string, total: integer}

GET /api/products

Search, filter, sort and page products.

Parameters: `q` (query) - Name, brand or series contains, `category_id` (query), `series_id` (query), `brand` (query) - Repeat for several: brand=Bandai&brand=SEGA, `min_price` (query), `max_price` (query), `in_stock` (query) - 1 = only in stock, `featured` (query) - 1 = only featured, `sort` (query), `page` (query) - Page number, from 1, `per_page` (query) - Rows per page (max 100 for admin lists, 50 for products).

Response 200: `data` = {items: list of `Product`, page: integer, per_page: integer, total: integer, total_pages: integer}

Errors: 422

GET /api/products/{product_id}

Product details with reviews and related products.

Parameters: `product_id` (path, required).

Response 200: `data` = `ProductDetail`

Errors: 404

Reviews

Star ratings and reviews; customers review products delivered to them

Method	Path	What it does	Login
`GET`	`/api/products/{product_id}/reviews`	Reviews of a product with the rating summary	-
`POST`	`/api/products/{product_id}/reviews`	Rate a product (posting again edits the review)	yes

Method	Path	What it does	Login
DELETE	/api/products/{product_id}/reviews/mine	Delete my review of this product	yes
GET	/api/reviews/to-review	Delivered products I haven't rated yet	yes

GET /api/products/{product_id}/reviews

Reviews of a product with the rating summary.

Login is optional; when logged in, the customer's own review comes first and has is_mine = true.

Parameters: `product_id` (path, required), `rating` (query) - Only this many stars (1-5), `with_comment` (query) - 1 = only reviews with a comment, `page` (query) - Page number, from 1, `per_page` (query) - Rows per page (max 100 for admin lists, 50 for products).

Response 200: `data` = {summary: `ReviewSummary`, items: list of `Review`, page: integer, per_page: integer, total: integer, total_pages: integer}

Errors: 404, 422

POST /api/products/{product_id}/reviews

Rate a product (posting again edits the review).

Only for products in one of the customer's delivered orders (403 otherwise). 200 "Review updated" when editing.

Parameters: `product_id` (path, required).

Body (application/json): {rating: integer, comment: string, tags: list of `quality` | `as_pictured` | `packaging` | `delivery` | `value`}

```
{
  "rating": 5,
  "comment": "Exactly like the photos!",
  "tags": [
    "quality",
    "packaging"
  ]
}
```

Response 201: `data` = {review: `Review`, summary: `ReviewSummary`}

Errors: 401, 403, 404, 422

DELETE /api/products/{product_id}/reviews/mine

Delete my review of this product.

Parameters: `product_id` (path, required).

Response 200: `data` = {summary: `ReviewSummary`}

Errors: 401, 404

GET /api/reviews/to-review

Delivered products I haven't rated yet.

Response 200: `data` = list of {product: `Product`, order_id: integer, order_number: string}

Errors: 401

Wishlist

Saved products

Method	Path	What it does	Login
`GET`	`/api/wishlist`	Saved products, newest first	yes
`POST`	`/api/wishlist/{product_id}`	Save a product	yes
`DELETE`	`/api/wishlist/{product_id}`	Remove a product	yes
`POST`	`/api/wishlist/batch-remove`	Remove several products	yes
`POST`	`/api/wishlist/batch-add`	Save several products again (Undo)	yes
`POST`	`/api/wishlist/move-to-cart`	Move products to the cart	yes

GET /api/wishlist

Saved products, newest first.

Response 200: `data` = {ids: list of integer, items: list of `Product`}

Errors: 401

POST /api/wishlist/{product_id}

Save a product.

Parameters: `product_id` (path, required).

Response 200: `data` = {product_id: integer, in_wishlist: boolean}

Errors: 401, 404

DELETE /api/wishlist/{product_id}

Remove a product.

Parameters: `product_id` (path, required).

Response 200: `data` = {product_id: integer, in_wishlist: boolean}

Errors: 401

POST /api/wishlist/batch-remove

Remove several products.

Body (application/json): {product_ids: list of integer}

```
{
  "product_ids": [
    3,
    7
  ]
}
```

Response 200: `data` = {ids: list of integer}

Errors: 401, 422

POST /api/wishlist/batch-add

Save several products again (Undo).

Body (application/json): {product_ids: list of integer}

```
{
  "product_ids": [
    3,
    7
  ]
}
```

Response 200: `data` = {ids: list of integer}

Errors: 401, 422

POST /api/wishlist/move-to-cart

Move products to the cart.

Adds one of each and takes it off the wishlist. Sold-out items stay (skipped).

Body (application/json): {product_ids: list of integer}

```
{
  "product_ids": [
    3,
    7
  ]
}
```

Response 200: `data` = {moved: list of integer, skipped: list of integer, ids: list of integer, cart_count: integer}

Errors: 401, 422

Cart

Shopping cart and promo codes

Method	Path	What it does	Login
`GET`	`/api/cart`	The cart with totals	yes
`POST`	`/api/cart`	Add to cart	yes

Method	Path	What it does	Login
DELETE	/api/cart	Empty the cart	yes
PUT	/api/cart/{item_id}	Change a line's quantity	yes
DELETE	/api/cart/{item_id}	Remove a line	yes
POST	/api/cart/coupon	Check a promo code against the cart	yes

GET /api/cart

The cart with totals.

Parameters: `coupon` (query) - Promo code to preview, e.g. WELCOME10.

Response 200: `data` = `Cart`

Errors: 401

POST /api/cart

Add to cart.

Adds to the line if the product is already in the cart. At most 20 of an item, and no more than the stock.

Body (application/json): {product_id: integer, quantity: integer, coupon: string}

```
{
  "product_id": 1,
  "quantity": 1
}
```

Response 201: `data` = `Cart`

Errors: 401, 404, 409, 422

DELETE /api/cart

Empty the cart.

Response 200: `data` = `Cart`

Errors: 401

PUT /api/cart/{item_id}

Change a line's quantity.

Parameters: `item_id` (path, required) - Cart line id.

Body (application/json): {quantity: integer, coupon: string}

```
{
  "quantity": 2
}
```

Response 200: `data` = `Cart`

Errors: 401, 404, 409, 422

DELETE /api/cart/{item_id}

Remove a line.

Parameters: `item_id` (path, required) - Cart line id, `coupon` (query) - Promo code to preview, e.g. WELCOME10.

Response 200: `data` = `Cart`

Errors: 401, 404

POST /api/cart/coupon

Check a promo code against the cart.

Body (application/json): {code: string}

```
{
  "code": "WELCOME10"
}
```

Response 200: `data` = `Cart`

Errors: 401, 422

Addresses

Delivery addresses

Method	Path	What it does	Login
GET	/api/addresses	Saved delivery addresses, default first	yes
POST	/api/addresses	Add an address	yes
PUT	/api/addresses/{address_id}	Change an address	yes
DELETE	/api/addresses/{address_id}	Delete an address	yes

GET /api/addresses

Saved delivery addresses, default first.

Response 200: `data` = list of `Address`

Errors: 401

POST /api/addresses

Add an address.

The first address, or one sent with is_default, becomes the default.

Body (application/json): `AddressInput`

Response 201: `data` = `Address`

Errors: 401, 422

PUT /api/addresses/{address_id}

Change an address.

Parameters: `address_id` (path, required).

Body (application/json): `AddressInput`

Response 200: `data` = `Address`

Errors: 401, 404, 422

DELETE /api/addresses/{address_id}

Delete an address.

If it was the default, the newest other address becomes the default.

Parameters: `address_id` (path, required).

Response 200: `data` = list of `Address`

Errors: 401, 404

Orders

Checkout, placing orders, order history

Method	Path	What it does	Login
`GET`	`/api/checkout`	Everything the Checkout screen shows	yes
`GET`	`/api/orders`	My orders, newest first, with their items	yes
`POST`	`/api/orders`	Place an order from the cart	yes
`GET`	`/api/orders/{order_id}`	One of my orders	yes
`POST`	`/api/orders/{order_id}/cancel`	Cancel my order	yes
`POST`	`/api/orders/{order_id}/received`	"Got it": I have the parcel	yes

GET /api/checkout

Everything the Checkout screen shows.

Parameters: `coupon` (query) - Promo code to preview, e.g. WELCOME10, `delivery` (query).

Response 200: `data` = `Cart` + {delivery_method: string, delivery_options: list of {id, label, detail, fee}, payment_methods: list of `khqr` | `aba` | `cod`, addresses: list of `Address`, store: {name, address, hours}}

Errors: 401

GET /api/orders

My orders, newest first, with their items.

Unpaid KHQR / ABA PayWay orders past their 15 minutes are cancelled first (stock goes back).

Response 200: `data` = list of `OrderDetail`

Errors: 401

POST /api/orders

Place an order from the cart.

Lowers stock and empties the cart. Store pickup is free and needs no address. 409 if an item sold out meanwhile or the payment method was turned off by the shop.

Body (application/json): {address_id: integer, delivery_method: `standard` | `pickup`, payment_method: `khqr` | `aba` | `cod`, coupon: string, note: string}

```
{
  "address_id": 1,
  "delivery_method": "standard",
  "payment_method": "khqr",
  "coupon": "WELCOME10",
  "note": ""
}
```

Response 201: `data` = `OrderDetail`

Errors: 401, 409, 422

GET /api/orders/{order_id}

One of my orders.

Parameters: `order_id` (path, required).

Response 200: `data` = `OrderDetail`

Errors: 401, 404

POST /api/orders/{order_id}/cancel

Cancel my order.

Only before it ships (pending or confirmed; see can_cancel). The stock goes back on sale. A paid KHQR / ABA PayWay order becomes refund_pending and the shop sends the money back.

Parameters: `order_id` (path, required).

Body (application/json): {reason: string}

```
{
  "reason": "Changed my mind"
}
```

Response 200: `data` = `OrderDetail`

Errors: 401, 404, 409, 422

POST /api/orders/{order_id}/received

"Got it": I have the parcel.

Only for a paid online order being delivered. 409 for store pickup and cash orders: the shop finishes those.

Parameters: `order_id` (path, required).

Response 200: `data` = `OrderDetail`

Errors: 401, 404, 409

Online payment

Online payment: Bakong KHQR and ABA PayWay

Method	Path	What it does	Login
`POST`	`/api/orders/{order_id}/khqr`	Make (once) the order's Bakong KHQR code	yes
`POST`	`/api/orders/{order_id}/payway`	Make (once) the order's ABA PayWay payment	yes
`GET`	`/api/orders/{order_id}/payment`	Payment status (polled by the app)	yes
`POST`	`/api/payments/payway/callback`	ABA PayWay pushback (called by PayWay, not the app)	-

POST /api/orders/{order_id}/khqr

Make (once) the order's Bakong KHQR code.

For payment_method khqr. Calling it again returns the same code. It is valid for 15 minutes.

Parameters: `order_id` (path, required).

Response 200: `data` = `PaymentState`

Errors: 401, 404, 409

POST /api/orders/{order_id}/payway

Make (once) the order's ABA PayWay payment.

For payment_method aba. Returns PayWay's KHQR and the ABA Mobile deeplink; calling it again returns the same ones. Valid for 15 minutes.

Parameters: `order_id` (path, required).

Response 200: `data` = `PaymentState`

Errors: 401, 404, 409, 502

GET /api/orders/{order_id}/payment

Payment status (polled by the app).

Asks Bakong or PayWay once: pending, paid or expired. Expired cancels the order.

Parameters: `order_id` (path, required).

Response 200: `data` = `PaymentState`

Errors: 401, 404

POST /api/payments/payway/callback

ABA PayWay pushback (called by PayWay, not the app).

Only a hint: the payment counts once PayWay's own check-transaction says APPROVED.

Body (application/x-www-form-urlencoded): {tran_id: string, status: string}

Response 200: Plain text OK

Profile

Profile, password, photo, notification choices

Method	Path	What it does	Login
PUT	/api/profile	Change name and phone	yes
GET	/api/profile/summary	Profile screen numbers: orders by status, points and tier	yes
PUT	/api/profile/password	Change password	yes
POST	/api/profile/avatar	Upload a profile photo	yes
GET	/api/profile/preferences	Settings > Notify me about	yes
PUT	/api/profile/preferences	Save notification choices	yes

PUT /api/profile

Change name and phone.

Body (application/json): {full_name: string, phone: string or null}

```
{
  "full_name": "Demo Customer",
  "phone": "012345678"
}
```

Response 200: `data` = `User`

Errors: 401, 422

GET /api/profile/summary

Profile screen numbers: orders by status, points and tier.

Response 200: `data` = `ProfileSummary`

Errors: 401, 404

PUT /api/profile/password

Change password.

Body (application/json): {current_password: string, new_password: string}

Response 200: `data` = {token: string}

Errors: 401, 422

POST /api/profile/avatar

Upload a profile photo.

JPG, PNG or WebP, 5 MB max. It must open as a picture; it is saved again as a WebP (at most 512 px).

Body (multipart/form-data): {file: string}

Response 200: `data` = `User`

Errors: 401, 422

GET /api/profile/preferences

Settings > Notify me about.

Response 200: `data` = `Preferences`

Errors: 401, 404

PUT /api/profile/preferences

Save notification choices.

Body (application/json): `Preferences`

```
{
  "notify_news": false
}
```

Response 200: `data` = `Preferences`

Errors: 401, 404, 422

Notifications

In-app notifications

Method	Path	What it does	Login
`GET`	`/api/notifications`	Newest 100 notifications and the unread count	yes

Method	Path	What it does	Login
GET	/api/notifications/unread-count	Unread count for the bell badge	yes
POST	/api/notifications/read-all	Mark all read	yes
POST	/api/notifications/{notification_id}/read	Mark one read	yes
DELETE	/api/notifications/{notification_id}	Delete one	yes

GET /api/notifications

Newest 100 notifications and the unread count.

Response 200: `data` = {items: list of `Notification`, unread: integer}

Errors: 401

GET /api/notifications/unread-count

Unread count for the bell badge.

Response 200: `data` = {unread: integer}

Errors: 401

POST /api/notifications/read-all

Mark all read.

Response 200: `data` = {unread: integer}

Errors: 401

POST /api/notifications/{notification_id}/read

Mark one read.

Parameters: `notification_id` (path, required).

Response 200: `data` = {unread: integer}

Errors: 401, 404

DELETE /api/notifications/{notification_id}

Delete one.

Parameters: `notification_id` (path, required).

Response 200: `data` = {unread: integer}

Errors: 401, 404

Contact

Questions to the shop

Method	Path	What it does	Login
GET	/api/contact	My Contact messages	yes
POST	/api/contact	Send a message to the shop	yes

GET /api/contact

My Contact messages.

Response 200: `data` = list of `ContactMessage`

Errors: 401

POST /api/contact

Send a message to the shop.

The order is kept only for order and payment questions and must be yours. The reply arrives as a `support` notification. At most 5 messages in 10 minutes and 30 a day per account (429).

Body (application/json): {topic: `order` | `payment` | `product` | `other`, message: string, order_id: integer}

```
{
  "topic": "order",
  "order_id": 12,
  "message": "When will my Luffy figure arrive?"
}
```

Response 201: `data` = `ContactMessage`

Errors: 401, 404, 422, 429

Admin

The shop team: a staff or admin account's token. Routes marked (admin only) need the owner's admin account

Method	Path	What it does	Login
GET	/api/admin/stats	Dashboard numbers, with paid revenue for each of the last 7 days	admin
GET	/api/admin/activity	Live activity: payments, new orders, messages, reviews, sold-out products	admin
GET	/api/admin/products	All products, hidden ones too	admin
POST	/api/admin/products	Add a product	admin
GET	/api/admin/products/{product_id}	One product	admin
PUT	/api/admin/	Change a product	admin

Method	Path	What it does	Login
	products/{product_id}	(send only what changes)	
DELETE	/api/admin/products/{product_id}	Hide a product from the shop	admin
POST	/api/admin/products/{product_id}/images	Add a photo	admin
DELETE	/api/admin/products/{product_id}/images/{image_id}	Remove a photo	admin
GET	/api/admin/categories	Categories with product counts	admin
POST	/api/admin/categories	Add a category	admin
PUT	/api/admin/categories/{category_id}	Rename or describe a category	admin
DELETE	/api/admin/categories/{category_id}	Delete an empty category	admin
GET	/api/admin/customers	Customers	admin
GET	/api/admin/customers/{user_id}	A customer with addresses and orders	admin
PUT	/api/admin/customers/{user_id}	Block or unblock a customer (admin only)	admin
GET	/api/admin/staff	Staff and admin accounts (admin only)	admin
POST	/api/admin/staff	Add a staff account (admin only)	admin
PUT	/api/admin/staff/{user_id}	Change, disable or reset a staff account (admin only)	admin
GET	/api/admin/orders	All orders, newest first	admin
GET	/api/admin/orders/{order_id}	One order with items and customer	admin
PUT	/api/admin/orders/{order_id}/status	Move an order on	admin
POST	/api/admin/orders/{order_id}/	Mark a cancelled paid order as refunded	admin

Method	Path	What it does	Login
	`refund`		
`GET`	`/api/admin/contact-messages`	Contact messages, newest first	admin
`POST`	`/api/admin/contact-messages/{message_id}/reply`	Answer a message	admin
`GET`	`/api/admin/coupons`	All promo codes, active first (admin only)	admin
`POST`	`/api/admin/coupons`	Add a promo code (admin only)	admin
`PUT`	`/api/admin/coupons/{coupon_id}`	Change a promo code (admin only; send only what changes)	admin
`DELETE`	`/api/admin/coupons/{coupon_id}`	Delete a promo code (admin only)	admin
`GET`	`/api/admin/payment-methods`	KHQR, ABA PayWay and cash: on or off (admin only)	admin
`PUT`	`/api/admin/payment-methods/{method}`	Turn a payment method on or off (admin only)	admin
`GET`	`/api/admin/reviews`	All reviews, newest first	admin
`DELETE`	`/api/admin/reviews/{review_id}`	Remove a review (spam, rude words)	admin

GET /api/admin/stats

Dashboard numbers, with paid revenue for each of the last 7 days.

Response 200: `data` = `Stats`

Errors: 401, 403

GET /api/admin/activity

Live activity: payments, new orders, messages, reviews, sold-out products.

Newest first. Each event says which action still applies (confirm, ship, reply, restock).

Parameters: `limit` (query) - How many (default 20, max 50).

Response 200: `data` = list of `ActivityEvent`

Errors: 401, 403

GET /api/admin/products

All products, hidden ones too.

Parameters: `q` (query) - Name or brand, `category_id` (query), `active` (query) - 1 = in the shop, 0 = hidden, `low_stock` (query) - 1 = 3 or fewer left, `page` (query) - Page number, from 1, `per_page` (query) - Rows per page (max 100 for admin lists, 50 for products).

Response 200: `data` = {items: list of `AdminProduct`, page: integer, per_page: integer, total: integer, total_pages: integer}

Errors: 401, 403

POST /api/admin/products

Add a product.

name, category_id and price are required. The slug is made from the name.

Body (application/json): `ProductInput`

```
{
  "name": "Roronoa Zoro 1/8 Scale Figure",
  "category_id": 1,
  "price": 120,
  "stock": 4
}
```

Response 201: `data` = `AdminProduct`

Errors: 401, 403, 422

GET /api/admin/products/{product_id}

One product.

Parameters: `product_id` (path, required).

Response 200: `data` = `AdminProduct`

Errors: 401, 403, 404

PUT /api/admin/products/{product_id}

Change a product (send only what changes).

Parameters: `product_id` (path, required).

Body (application/json): `ProductInput`

```
{
  "stock": 10,
  "discount_price": null
}
```

Response 200: `data` = `AdminProduct`

Errors: 401, 403, 404, 422

DELETE /api/admin/products/{product_id}

Hide a product from the shop.

Products are hidden, not deleted, because past orders point to them. Send is_active: true to bring one back.

Parameters: `product_id` (path, required).

Response 200: `data` = `AdminProduct`

Errors: 401, 403, 404

POST /api/admin/products/{product_id}/images

Add a photo.

JPG, PNG or WebP, 5 MB max. Made square and saved as WebP. The first photo is the main one.

Parameters: `product_id` (path, required).

Body (multipart/form-data): {file: string}

Response 201: `data` = `AdminProduct`

Errors: 401, 403, 404, 422

DELETE /api/admin/products/{product_id}/images/{image_id}

Remove a photo.

Parameters: `product_id` (path, required), `image_id` (path, required).

Response 200: `data` = `AdminProduct`

Errors: 401, 403, 404

GET /api/admin/categories

Categories with product counts.

Response 200: `data` = list of `AdminCategory`

Errors: 401, 403

POST /api/admin/categories

Add a category.

Body (application/json): {name: string, description: string}

```
{
  "name": "Plush & Merch",
  "description": "Plushies, keychains, acrylic stands"
}
```

Response 201: `data` = `AdminCategory`

Errors: 401, 403, 422

PUT /api/admin/categories/{category_id}

Rename or describe a category.

Parameters: `category_id` (path, required).

Body (application/json): {name: string, description: string}

Response 200: `data` = `AdminCategory`

Errors: 401, 403, 404, 422

DELETE /api/admin/categories/{category_id}

Delete an empty category.

409 while it still has products.

Parameters: `category_id` (path, required).

Response 200: `data` = any or null

Errors: 401, 403, 404, 409

GET /api/admin/customers

Customers.

Parameters: `q` (query) - Name, email or phone, `active` (query) - 1 = active, 0 = blocked, `page` (query) - Page number, from 1, `per_page` (query) - Rows per page (max 100 for admin lists, 50 for products).

Response 200: `data` = {items: list of `Customer`, page: integer, per_page: integer, total: integer, total_pages: integer}

Errors: 401, 403

GET /api/admin/customers/{user_id}

A customer with addresses and orders.

Parameters: `user_id` (path, required).

Response 200: `data` = `CustomerDetail`

Errors: 401, 403, 404

PUT /api/admin/customers/{user_id}

Block or unblock a customer (admin only).

A blocked customer cannot log in, and their current token stops working (403).

Parameters: `user_id` (path, required).

Body (application/json): {is_active: boolean}

```
{
  "is_active": false
}
```

Response 200: `data` = `Customer`

Errors: 401, 403, 404, 422

GET /api/admin/staff

Staff and admin accounts (admin only).

Admins first, then by name.

Parameters: `q` (query) - Name, email or phone, `active` (query) - 1 = active, 0 = disabled.

Response 200: `data` = list of `Staff`

Errors: 401, 403

POST /api/admin/staff

Add a staff account (admin only).

role is staff (default) or admin. Give them the password; they can change it in Profile.

Body (application/json): {full_name: string, email: string, password: string, phone: string or null, role: `staff` | `admin`}

```
{
  "full_name": "Sokha Chan",
  "email": "sokha@shop.com",
  "password": "welcome123",
  "role": "staff"
}
```

Response 201: `data` = `Staff`

Errors: 401, 403, 422

PUT /api/admin/staff/{user_id}

Change, disable or reset a staff account (admin only).

Send only what changes. A new role, a new password or disabling signs them out everywhere. 409 when you try to change your own role, disable yourself, or leave the shop without an active admin.

Parameters: `user_id` (path, required).

Body (application/json): `StaffInput`

```
{
  "is_active": false
}
```

Response 200: `data` = `Staff`

Errors: 401, 403, 404, 409, 422

GET /api/admin/orders

All orders, newest first.

Parameters: `status` (query), `refund` (query) - 1 = cancelled paid orders whose refund is still to send, `q` (query) - Order number, customer name or email, `page` (query) - Page number, from 1, `per_page` (query) - Rows per page (max 100 for admin lists, 50 for products).

Response 200: `data` = {items: list of `AdminOrder`, page: integer, per_page: integer, total: integer, total_pages: integer}

Errors: 401, 403, 422

GET /api/admin/orders/{order_id}

One order with items and customer.

Parameters: `order_id` (path, required).

Response 200: `data` = `AdminOrderDetail`

Errors: 401, 403, 404

PUT /api/admin/orders/{order_id}/status

Move an order on.

Steps: pending -> confirmed -> shipped -> delivered, or cancelled from pending/confirmed. An unpaid KHQR order can only be cancelled. Cancelling puts the stock back, and a paid online order becomes refund_pending. The customer gets a notification for each step.

Parameters: `order_id` (path, required).

Body (application/json): {status: `confirmed` | `shipped` | `delivered` | `cancelled`, reason: string}

```
{
  "status": "shipped"
}
```

Response 200: `data` = `AdminOrderDetail`

Errors: 401, 403, 404, 409

POST /api/admin/orders/{order_id}/refund

Mark a cancelled paid order as refunded.

After the shop has sent the money back (bank transfer or ABA). 409 when no refund is waiting. The customer is notified.

Parameters: `order_id` (path, required).

Response 200: `data` = `AdminOrderDetail`

Errors: 401, 403, 404, 409

GET /api/admin/contact-messages

Contact messages, newest first.

Parameters: `status` (query), `page` (query) - Page number, from 1, `per_page` (query) - Rows per page (max 100 for admin lists, 50 for products).

Response 200: `data` = {items: list of `ContactMessage` + {customer}, page: integer, per_page: integer, total: integer, total_pages: integer}

Errors: 401, 403

POST /api/admin/contact-messages/{message_id}/reply

Answer a message.

The customer gets the reply in Notifications.

Parameters: `message_id` (path, required).

Body (application/json): {reply: string}

```
{
  "reply": "Your figure ships tomorrow!"
}
```

Response 200: `data` = `ContactMessage`

Errors: 401, 403, 404, 422

GET /api/admin/coupons

All promo codes, active first (admin only).

Response 200: `data` = list of `AdminCoupon`

Errors: 401, 403

POST /api/admin/coupons

Add a promo code (admin only).

Code: letters and numbers (stored upper case). A percent coupon is at most 100.

Body (application/json): `CouponInput`

```
{
  "code": "ANIME5",
  "description": "$5 off any order",
  "discount_type": "fixed",
  "value": 5
}
```

Response 201: `data` = `AdminCoupon`

Errors: 401, 403, 422

PUT /api/admin/coupons/{coupon_id}

Change a promo code (admin only; send only what changes).

Parameters: `coupon_id` (path, required).

Body (application/json): `CouponInput`

```
{
  "is_active": false
}
```

Response 200: `data` = `AdminCoupon`

Errors: 401, 403, 404, 422

DELETE /api/admin/coupons/{coupon_id}

Delete a promo code (admin only).

Parameters: `coupon_id` (path, required).

Response 200: `data` = any or null

Errors: 401, 403, 404

GET /api/admin/payment-methods

KHQR, ABA PayWay and cash: on or off (admin only).

Response 200: `data` = list of `AdminPaymentMethod`

Errors: 401, 403

PUT /api/admin/payment-methods/{method}

Turn a payment method on or off (admin only).

Turned-off methods are hidden at checkout and refused by Place order; orders already placed with one can still be paid. 409 when it is the last method still on.

Parameters: `method` (path, required).

Body (application/json): {enabled: boolean}

```
{
  "enabled": false
}
```

Response 200: `data` = list of `AdminPaymentMethod`

Errors: 401, 403, 404, 409, 422

GET /api/admin/reviews

All reviews, newest first.

Parameters: `rating` (query) - Only this many stars, `q` (query) - Product or customer name, `page` (query) - Page number, from 1, `per_page` (query) - Rows per page (max 100 for admin lists, 50 for products).

Response 200: `data` = {items: list of `AdminReview`, page: integer, per_page: integer, total: integer, total_pages: integer}

Errors: 401, 403

DELETE /api/admin/reviews/{review_id}

Remove a review (spam, rude words).

Parameters: `review_id` (path, required).

Response 200: `data` = any or null

Errors: 401, 403, 404

Data shapes

Envelope

{success: boolean, message: string, data: any}

Error

{success: boolean, message: string, errors: object or null}

User

{id: integer, full_name: string, email: string, phone: string or null, avatar_url: string or null, role: `customer` | `staff` | `admin`, created_at: string}

Category

{id: integer, name: string, slug: string, description: string or null, image_url: string or null, product_count: integer, cover_image: string or null}

Series

{id: integer, name: string, slug: string, image_url: string or null, product_count: integer, cover_image: string or null}

Product

{id: integer, category: `Category`, series: `Series` or null, brand: string or null, name: string, slug: string, description: string or null, price: number, discount_price: number or null, stock: integer, is_featured: boolean, image_url: string or null, images: list of string, rating: number, review_count: integer}

Review

{id: integer, product_id: integer, user: string, rating: integer, comment: string or null, tags: list of `quality` | `as_pictured` | `packaging` | `delivery` | `value`, created_at: string, edited: boolean, is_mine: boolean}

ReviewSummary

{rating: number, count: integer, stars: object, with_comment: integer, tags: object}

ProductDetail

`Product` + {reviews: list of `Review`, review_summary: `ReviewSummary`, my_review: `Review` or null, can_review: boolean, related: list of `Product`}

CartItem

{id: integer, quantity: integer, subtotal: number, product: `Product`}

Totals

{count: integer, subtotal: number, discount: number, shipping_fee: number, free_shipping_min: number, total: number, savings: number, coupon: {code, description, discount}, coupon_error: string or null}

Cart

{items: list of `CartItem`} + `Totals`

Address

{id: integer, label: string, recipient_name: string, phone: string, street: string, city: string, province: string or null, postal_code: string or null, is_default: boolean, one_line: string}

AddressInput

{label: string, recipient_name: string, phone: string, street: string, city: string, province: string, postal_code: string, is_default: boolean}

OrderItem

{product_id: integer or null, product_name: string, image_url: string or null, category: `Category` or null, unit_price: number, quantity: integer, subtotal: number, my_rating: integer or null}

Order

{id: integer, order_number: string, status: `pending` | `confirmed` | `shipped` | `delivered` | `cancelled`, payment_method: `cod` | `khqr` | `aba`, payment_status: `unpaid` | `paid` | `failed` | `refund_pending` | `refunded`, subtotal: number, shipping_fee: number, discount: number, coupon_code: string or null, total: number, delivery_method: `standard` | `pickup`, item_count: integer, created_at: string, can_cancel: boolean, cancelled_by: `customer` | `shop` | `system`, cancel_reason: string or null, cancelled_at: string or null, refunded_at: string or null}

OrderDetail

`Order` + {shipping: {name, phone, address}, note: string or null, items: list of `OrderItem`}

Payment

{provider: `bakong` | `payway`, qr: string, deeplink: string or null, amount: number, currency: string, status: `pending` | `paid` | `expired`, expires_at: string, paid_at: string or null}

PaymentState

{payment: `Payment`, order: `OrderDetail`}

Notification

{id: integer, title: string, message: string, type: `order` | `promo` | `system`, event: `placed` | `paid` | `confirmed` | `shipped` | `delivered` | `cancelled` | `welcome` | `promo` | `news` | `support`, is_read: boolean, order: {id, number}, image_url: string or null, promo_code: string or null, created_at: string}

ContactMessage

{id: integer, topic: `order` | `payment` | `product` | `other`, message: string, status: `open` | `answered`, reply: string or null, order: {id, number}, created_at: string, replied_at: string or null}

Preferences

{notify_orders: boolean, notify_offers: boolean, notify_news: boolean}

ProfileSummary

{user: `User`, stats: {orders, wishlist, reviews}, order_status: {to_pay, to_ship, to_receive, to_review}, points: integer, tier: {name, next_name, points_to_next, progress}}

AdminProduct

`Product` + {is_active: boolean, photos: list of {id, url, is_primary}, created_at: string}

ProductInput

{name: string, category_id: integer, series_id: integer or null, brand: string, description: string, price: number, discount_price: number or null, stock: integer, is_featured: boolean, is_active: boolean}

AdminCategory

`Category` + {product_count: integer}

Customer

`User` + {is_active: boolean, order_count: integer, total_spent: number}

Staff

`User` + {is_active: boolean, last_login_at: string or null}

StaffInput

{full_name: string, phone: string or null, role: `staff` | `admin`, is_active: boolean, password: string}

CustomerDetail

`Customer` + {addresses: list of `Address`, orders: list of `Order`}

AdminOrder

`Order` + {customer: {id, full_name, email, phone}, next_statuses: list of string, image_url: string or null}

AdminOrderDetail

`OrderDetail` + `AdminOrder`

AdminCoupon

{id: integer, code: string, description: string, discount_type: `percent` | `fixed`, value: number, min_subtotal: number, max_discount: number or null, expires_at: string or null, is_active: boolean, used: integer}

CouponInput

{code: string, description: string, discount_type: `percent` | `fixed`, value: number, min_subtotal: number, max_discount: number or null, expires_at: string or null, is_active: boolean}

AdminPaymentMethod

{id: `khqr` | `aba` | `cod`, label: string, enabled: boolean, set_up: boolean}

AdminReview

`Review` + {product: {id, name, image_url}, customer: {id, full_name, email}}

Stats

{week: list of {date, revenue}, week_revenue: number, products: integer, low_stock: integer, categories: integer, customers: integer, orders: integer, orders_to_handle: integer, refunds_to_send: integer, revenue: number, open_messages: integer}

ActivityEvent

{id: string, kind: `paid` | `order` | `cancelled` | `message` | `review` | `sold_out`, at: string, title: string, message: string, image_url: string or null, action: `confirm` | `ship` | `reply` | `restock` | `refund`, status: string or null, order_id: integer or null, order_number: string or null, message_id: integer or null, product_id: integer or null}